

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Decide on implementation strategy for the GBrain rewrite / Hermes enhancements (Rust/Go, MemPalace + SQLite native).
- Options:
 1. Build in a worktree branch and wait for Node D (dedicated Hermes Pod).
 2. Implement a **super optimized and lightweight version** in the current tri-node system now, then expand capability once Node D is online.
- Focus on maintaining stability, sovereignty, zero-trust, and performance while delivering immediate value as a private self-improving personal intelligence assistant.
- Maintain: Node A and Node B completely unchanged in core logic.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (Current Stack Constraints + Best Practices)

Current tri-node (A = Mooncake KV buffer, B = Director Q8, C = Oracle Q4 + Omi + Hermes + 500 GB SSD) is already handling voice, screen awareness, RKG, and self-healing. Hermes is co-located on Node C and can absorb a lightweight GBrain engine with proper tuning. Delaying until Node D risks losing momentum and testing feedback. Branching is cheap in Git and aligns with Nix flake workflows for safe experimentation.

RECOMMENDATION

Implement a super optimized, lightweight version in the current tri-node system now (on a feature branch), then expand power and capability once Node D arrives.

Why this is the correct power move:

- **Immediate Value & Testing:** You get real-world feedback on the GBrain rewrite (voice note ingestion, relational graph, Dream Cycle) while using the system daily. This de-risks the full implementation.

- **Tri-Node Can Handle Lite Version:** With aggressive tuning (smaller GEPA population, disk spilling on Node C’s 500 GB SSD, Mooncake offload from Node A), Hermes + GBrain will run comfortably on Node C without degrading voice or reasoning performance.
- **Clean Expansion Path:** When Node D arrives, you simply move the full Hermes/GBrain workload to the dedicated pod via declarative Nix role switch. The tri-node version becomes the “lite mode” fallback.
- **Branch Safety:** Work in a `feature/gbrain-rewrite` branch. Merge to main only when stable. This keeps your production mesh untouched.

PROS / CONS

Approach	Pros	Cons	Verdict
Wait for Node D + branch	Clean separation, full power from day 1	Delayed value, no real testing feedback	Suboptimal
Implement lite now on tri-node	Immediate usability, iterative testing, smooth upgrade path	Requires careful tuning on Node C	Recommended

STEP-BY-STEP IMPLEMENTATION PLAN (Optimized Lite Version on Tri-Node)

1. Create Feature Branch

```
Bash
git checkout -b feature/gbrain-rewrite
```

2. Lite GBrain Engine (Rust Crate)

- Minimal viable: SQLite + sqlite-vec hybrid search, basic graph (nodes + edges), git sync, Dream Cycle batch job.
- Run as a service on Node C (alongside current Hermes).
- Offload heavy context to Mooncake on Node A.
- Use the 500 GB SSD on Node C for graph storage and audit ledger.

3. Hermes Integration

- Hermes on Node C calls the lite GBrain for enrichment and storage.
- Keep current voice pipeline intact; add GBrain ingestion as an optional post-processing step.

4. Tuning for Tri-Node (Lite Mode)

- GEPA: population=8, trajectories=60, run every 12 min.
- Audit: smaller batches, 60s interval.
- Ray: limit to 6–8 workers on Node C.
- Disk spilling enabled for safety.

5. Testing & Validation

- Daily voice notes → GBrain ingestion → Dream Cycle → visible in MemPalace.
- Monitor Node C VRAM/CPU/SSD usage with Sovereign Pulse.

6. Node D Expansion (Future)

- When Node D arrives, move full GBrain + heavy Hermes workloads to it via Nix role switch.
- Tri-node becomes “lite mode” automatically.

ARCHITECT SIGN-OFF

Implementing a super optimized, lightweight GBrain rewrite on the current tri-node system now (on a feature branch) is the correct strategy. It delivers immediate value and real testing feedback while keeping the system stable. The 500 GB SSD on Node C and Mooncake on Node A provide enough breathing room for a tuned lite version. When Node D (Hermes Pod) arrives, you expand seamlessly to full power without disruption. This approach respects your current hardware constraints, maintains full sovereignty, and positions you for a smooth quad-node upgrade. The rewrite deepens Dreaming and knowledge capabilities without rushing the architecture.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the starter Rust crate structure (`sovereign-gbrain`) with lite SQLite + MemPalace integration for the tri-node branch?
2. Shall I provide the Nix module updates for running the lite GBrain on Node C?

Reply with your priority and I will deliver the concrete starter files. The GBrain integration path is ready.